Skins framework: building and running multiple skins

Nov 7, 2026 · @Chris

Build the skin system once, before any one skin, so Classic, Sleeve (Idea B) and Timeline (Idea A) all switch the same way, share the same data and are tested the same way. This framework replaces phase 1 of both skin guides. After it lands, each skin is only its own CSS, a few slot components and its screens.

The prompt

Export this doc as Markdown and save it as `docs/design/skins-framework.md`, next to `sleeve-skin.md` and `timeline-skin.md`. Then give the coding agent this prompt:

```text
You're working in the audio-scrobbler-app repo. Before building any new visual skin, I want a skins framework so the app can offer several looks: Classic (today's design, the default), Sleeve and Timeline. The spec is docs/design/skins-framework.md; the skin specs are docs/design/sleeve-skin.md and docs/design/timeline-skin.md. Read all three before planning.

This task is the framework only, with no new screens:
1. Follow AGENTS.md: read HANDOVER.md, plan with agents/planner_agent.md, and wait for my approval before writing code. Use the safe-change process.
2. Build the skin registry, preference, data-skin attribute, no-flash script, ?skin= preview, Settings control, useSkin() hook, Slot pattern, token contract and the test matrix, exactly as the framework spec describes.
3. Register Classic only. Sleeve and Timeline get empty CSS files and are hidden from Settings until their first screen lands.
4. Classic must stay pixel-identical. Prove it with before and after screenshots of every route at 390 px and 1280 px.
5. Add a "Skins" section to DESIGN.md that points to these specs.

If the spec doesn't fit the code, stop and ask rather than guessing.
```

Architecture

A skin is a named bundle of three things: CSS tokens, a few slot components, and metadata. The app chooses one skin per user, and the skin sits on top of the existing theme and accent settings rather than replacing them.

Piece	What it does	Where
Registry	One list of skins, each with an id, a display name, `supportsThemes` (true for Classic and Timeline, false for Sleeve, which is dark only), `status` (`hidden`, `preview` or `live`) and its slot components	`frontend/src/skins/index.js`
Preference	`skin` column with a check constraint listing the registry ids, default `'classic'`. Unknown values fall back to Classic.	`backend/app/models.py`, Alembic migration, `schemas/preferences.py`, `preferences_service.py`
Attribute	`<html data-skin="classic">`, set beside the existing `data-theme` and `data-accent`. When a skin has `supportsThemes: false`, also

force `data-theme="dark"` while it is active, and restore the saved theme when switching back. | `App.jsx`, `SettingsPage.jsx` |
No flash	An inline script in `index.html` reads `audio-scrobbler-skin` from `localStorage` and sets the attribute before the CSS paints	`frontend/index.html`
Preview	`?skin=<id>` overrides the preference for the current session, including `hidden` skins, for development and screenshots. `?skin=` with no value clears it.	`App.jsx`
Hook	`useSkin()` returns `{ id, skin, setSkin }` and re-renders when the skin changes	`frontend/src/skins/useSkin.js`
Settings	"Skin" control in Appearance listing the `live` skins, plus `preview` skins for the owner. Each option shows a small thumbnail.	`SettingsPage.jsx`

Use this file layout:

```text
frontend/src/skins/
  index.js           registry
  useSkin.js         hook + context
  Slot.jsx           slot helper
  tokens.contract.js  list of required tokens (used by tests)
  classic/           Classic needs no CSS here: styles.css is Classic
  sleeve/   sleeve.css   slots/*.jsx
  timeline/ timeline.css slots/*.jsx
```

Every selector in a skin's CSS starts with `[data-skin="<id>"]`. Import all skin CSS once from `main.jsx`; scoped rules cost nothing when inactive. Keep Classic as the unscoped base in `styles.css`, so a missing rule in a new skin falls back to Classic instead of breaking.

The token contract

Shared components should read only semantic tokens, never raw hex values or skin-specific names. Every skin then sets the same short list, and a component styled once works in every skin. Map the existing `--color-*` variables onto this contract; don't run a second naming scheme beside them.

Token	Meaning	Classic value today
`--color-canvas`	page background	`#F7F7F7`
`--color-surface`	cards, sheets, tab bar	`#FFFFFF`
`--color-surface-raised`	search fields, pills, avatar	`#FFFFFF`
`--color-ink`	primary text	`#111111`
`--color-muted`	secondary text, inactive icons	`#707070`
`--color-border` / `--color-border-subtle`	dividers, outlines	`#E5E5E5` / `#F0F0F0`
`--color-primary` / `--color-accent`	the user's accent: active states, markers	per `data-accent`
`--color-on-accent`	text on an accent fill	new: white or ink, whichever has more contrast
`--color-focus`	focus ring	accent

`--color-danger`, `--color-success`, `--color-warning` (+ `-soft`)	status	as today
`--font-display` / `--font-body`	heading and UI faces	new: Inter / Inter
`--radius-sm` / `--radius-md` / `--radius-pill`	corners	8 / 12 px / new: 999 px
`--space-gutter`	page side padding	new: 16 px
`--nav-height`	bottom tab bar height, 0 when the skin has none	new: 0

A skin may add private tokens (`--sl-*`, `--tl-*`), but only its own CSS and slots may read them. `tokens.contract.js` lists the contract, and a test fails if any registered skin leaves one unset.

Component strategy

Pick the lightest tool that does the job, in this order:

1. **Tokens only.** Colour, type, radius and spacing changes need no new code. Most of a skin should be this.
2. **Scoped CSS.** Layout tweaks to an existing component, under `[data-skin="<id>"] .component`. Namespace by page too when a shared component is styled for one page.
3. **A slot.** When a skin needs different markup in one place, the page renders `<Slot name="todayHero" {...data} />`. The registry maps each slot name to a component per skin, and anything not mapped falls back to the Classic component. Typical slots are `appNav` (header links versus a bottom tab bar), `subpageHeader`, `todayHero` and `playPage`.
4. **A whole page.** Avoid this. If a skin seems to need it, split the page into slots instead.

Three rules keep skins from drifting apart:

- **Data lives outside the skin.** Hooks and API calls stay in the page component, and slots receive props. A slot never fetches.
- **Same routes, same features.** Every skin reaches every route. A skin can arrange things differently but never hides a feature. A skin may add a route (like `/play`) only if other skins can link to it too.
- **Protected components stay protected.** The Retro Jukebox is the same in every skin. List any other protected components in the registry so tests can check they're untouched.

Testing

Every skin multiplies the looks the app can have: 3 skins × 3 themes × 8 accents. Test the combinations automatically, not by eye.

Test	What it checks	Runs
Token contract	Every registered skin sets every contract token, for each theme it supports	unit, every PR
Contrast matrix	For each skin, theme and accent: ink and muted on canvas and surface ≥ 4.5:1; on-accent text ≥ 4.5:1; accent as a marker or border ≥ 3:1 against canvas	unit, every PR
Flat rules	No gradients, and no shadows except focus rings, in every skin CSS file, the same as `cardConsistency.test.js` checks for `styles.css`	unit, every PR
Scope check	Every selector in `skins/<id>/*.css` starts with `[data-skin="<id>"]`	unit, every PR
Protected files	Retro Jukebox files have no changes against `main`	CI, every PR

Classic regression	Playwright screenshots of every route at 390 px and 1280 px with `?skin=classic`, compared with baselines from `main`	CI, every PR
Skin snapshots	The same routes for each `preview` or `live` skin, in light and dark where supported	CI, every PR; update baselines on purpose only
Switch behaviour	Changing skin in Settings applies without a reload, survives a reload, and switching back restores theme and accent	e2e

For the contrast matrix, compute the values in the test from the same formulas the CSS uses, including `color-mix`. Testing a hand-copied hex list goes stale.

Performance and accessibility

Performance.

- Self-host fonts with `@fontsource`, subset to Latin, and import only the weights a skin uses. A `@font-face` downloads only when a rendered element uses it, so inactive skins' fonts cost nothing.
- Keep each skin's CSS small: tokens and overrides, not a copy of `styles.css`. Aim for under 15 KB gzipped per skin.
- Lazy-load slot components with `React.lazy` so Classic users don't download other skins' markup.
- Record bundle size and the time until the first content appears on the preview server before the framework lands, and after each skin.

Accessibility. These apply to every skin, and the tests above enforce most of them:

- Text contrast is at least 4.5:1, and indicators such as markers, active bars and outlines are at least 3:1.
- Every tap target is at least 44 × 44 px.
- Focus is always visible: a 2 px ring in `--color-focus` with an offset.
- Meaning never rests on colour alone. Pair it with a shape, a label or position.
- Respect `prefers-reduced-motion` for any skin transition.
- Navigation is a `<nav>` with `aria-current="page"` on the active item, whether it sits in a header or a tab bar.
- The skin picker is a labelled radio group, not a set of unlabelled thumbnails.

Adding a new skin

Follow these steps for each new skin. Sleeve and Timeline both follow this path.

1. Write a skin spec in `docs/design/<id>-skin.md`, covering its tokens, slots, screens and open decisions.
2. Add the skin to the registry as `hidden`, with an empty `skins/<id>/<id>.css` file.
3. Fill the token contract for every theme the skin supports, and make the contrast matrix pass.
4. Add slot components one at a time, smallest structural change first. Navigation usually comes first.
5. Restyle screens one per PR, checking each against the mockups at 390 px with `?skin=<id>`.
6. Set `status: "preview"` so the owner can choose it in Settings and live with it for a while.
7. Add skin snapshot baselines to CI.
8. Set `status: "live"` when every route is done and the owner signs off.

Each PR should keep these true:

- [ ] Classic regression screenshots are unchanged.
- [ ] The contract, contrast, flat-rule and scope tests pass.
- [ ] Protected components are untouched.
- [ ] There are screenshots of touched pages in the new skin, in each supported theme.
- [ ] `npm run lint && npm test && npm run build` passes.

Governance

- **One source of truth per skin.** `DESIGN.md` describes Classic and links to each skin's spec. A rule that applies to every skin, such as Flat-By-Default, stays in `DESIGN.md` and the skin specs don't repeat it.
- **The default stays Classic** until the owner chooses otherwise. Changing the default is its own PR, with a note in the changelog.
- **New features ship in every live skin at once.** A feature PR includes the styling for each live skin, or the feature waits. Write this rule into `agents/implementation_agent.md` so agents follow it too.
- **Cap the number of skins.** Each live skin adds review and test cost to every future change. Three is plenty for a personal app. Before adding a fourth, retire one.
- **Retiring a skin:** set it to `hidden`, migrate anyone using it to Classic and show them a one-off notice, then delete its folder and registry entry after a release.